

Full length article

Uncovering Bluetooth vulnerabilities with binary coverage-guided fuzz testing and controller emulation

Zhao Min Chen^{a,b}, Tien-Chih Lin^b, Guan-Yan Yang^a, Yu-Sheng Lin^c, Farn Wang^a, Kuo-Hui Yeh^{d,e,*}

^a Department of Electrical Engineering, National Taiwan University, No. 1, Sec. 4, Roosevelt Rd., Taipei, 106319, Taiwan, ROC

^b CyCraft Technology, 6F., No. 3, Yuandong Rd., Banqiao Dist., New Taipei City, 220056, Taiwan, ROC

^c Department of Computer Science at National Yang Ming Chiao Tung University, No. 1001, Daxue Rd., East Dist., Hsinchu City, 300093, Taiwan, ROC

^d Institute of Artificial Intelligence Innovation, National Yang Ming Chiao Tung University, No. 1001, Daxue Rd., East Dist., Hsinchu City, 300093, Taiwan, ROC

^e Department of Information Management, National Dong Hwa University, No. 1, Sec. 2, Da Hsueh Rd. Shoufeng, Hualien, 974301, Taiwan, ROC

ARTICLE INFO

Keywords:

Bluetooth

Fuzzing

Wireless security

ABSTRACT

Bluetooth Basic Rate and Enhanced Data Rate (BR/EDR) is a typical wireless transmission technology in embedded devices and personal computers. Recently, some Bluetooth fuzzing studies have been conducted to identify vulnerabilities in devices and chips; however, they primarily focus on low-level aspects of the Bluetooth stack, and many rely on physical devices to deliver packets. This paper proposes FuBuKi, a binary coverage-guided Bluetooth fuzzer, to detect Bluetooth vulnerabilities on emulated Bluetooth controllers. FuBuKi can generate and mutate the input with binary code coverage by constructing the corresponding input transformer. Furthermore, FuBuKi can emulate the real-world target binaries and successfully discover crashes with a dedicated input transformer. We applied FuBuKi on three different types of Bluetooth stacks with five different profiles to verify the effectiveness. The results of the experiment confirm that FuBuKi is also able to fuzz profiles that require pairing and bonding mechanisms, compared to other state-of-the-art work.

1. Introduction

In this section, we will present the background, motivation, contribution, and organization of this paper.

1.1. Background

The introduction of Bluetooth technology has significantly enhanced the convenience of our daily lives. Bluetooth has become an integral part of modern society, from the seamless connectivity of wireless headphones to the effortless control of smart home devices. This ubiquitous technology is embedded in many devices, from personal gadgets like smartphones and fitness trackers to more complex systems such as smart appliances and electric vehicles. With the proliferation of Bluetooth-enabled devices, there has been a corresponding increase in discussions surrounding the security implications of this technology. In 2017, [Seri and Vishnepolsky \(2017\)](#) proposed BlueBorne, a critical

set of flaws affecting Bluetooth implementations across all major operating systems, including Windows, iOS, and Android. In 2020, Andy Nguyen unveiled BleedingTooth ([BleedingTooth, 2024](#)), a collection of zero-click vulnerabilities specifically targeting the Linux Bluetooth stack, which allows attackers to compromise a Linux-based Bluetooth-activated device without any physical interaction. In 2023, [Childs \(2024\)](#) used Bluetooth as initial access to take down Tesla in Pwn2Own Vancouver 2023. These vulnerabilities indicate the impact of Bluetooth vulnerabilities on the security landscape, showcasing how critical flaws can lead to severe consequences across various platforms.

1.2. Motivation

As aforementioned, the vulnerabilities in Bluetooth have a high security impact. Several studies have been conducted to detect unknown Bluetooth vulnerabilities. Based on data from the CVE database for Linux Bluetooth vulnerabilities ([CVE, 2024a,b](#)), as shown in [Fig. 1](#), the

* Corresponding author at: Institute of Artificial Intelligence Innovation, National Yang Ming Chiao Tung University, No. 1001, Daxue Rd., East Dist., Hsinchu City, 300093, Taiwan, ROC.

E-mail addresses: jim.chen@cyccraft.com (Z.M. Chen), dange.lin@cyccraft.com (T. Lin), f11921091@ntu.edu.tw (G. Yang), t03.c@nycu.edu.tw (Y. Lin), farn@ntu.edu.tw (F. Wang), khyeh@nycu.edu.tw (K. Yeh).

URLs: <https://ianyang66.github.io> (G. Yang), <https://cc.ee.ntu.edu.tw/~farn/> (F. Wang), <https://khyeh.web.nycu.edu.tw/> (K. Yeh).

<https://doi.org/10.1016/j.cose.2026.104929>

Received 27 July 2025; Received in revised form 11 March 2026; Accepted 16 April 2026

Available online 25 April 2026

0167-4048/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

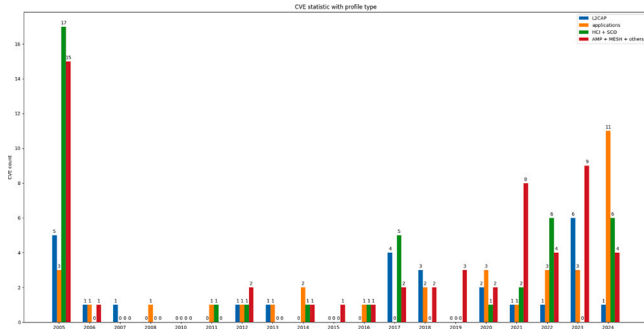


Fig. 1. Statistical results of CVE related to Linux Bluetooth in profiles and years (CVE, 2024a,b).

number of vulnerabilities has increased significantly over recent years. Application profiles, including AVRCP, PBAP, and SDP, have seen the most substantial growth in vulnerability counts. Consequently, it is imperative to discover vulnerabilities automatically. However, existing studies have some limitations:

1. Unable to detect bugs underlying high levels of Bluetooth stack (e.g, L2Fuzz Park et al., 2022)
2. Require a physical device, source code, or enable some specific features for instrumentation (e.g, SweenTooth Garbelini et al., 2020, VirtFuzz Huster et al., 2024)
3. Remain low or unknown in overall coverage (e.g, ToothPicker Heinze et al., 2020).

1.3. Contribution

We propose FuBuKi, a fuzzer based on AFL++ (Fioraldi et al., 2020), specifically designed for Bluetooth Basic Rate/Enhanced Data Rate (BR/EDR) stacks to address these shortcomings. FuBuKi has a chip emulator component, enabling users to utilize advanced Bluetooth development frameworks to craft testing harnesses and fuzz various target Bluetooth stacks, even without access to their source code, over a virtualized connection. Furthermore, benefiting from its architectural design, FuBuKi supports the simultaneous operation of multiple fuzzer instances, enhancing its efficiency and effectiveness. To demonstrate the efficacy of FuBuKi, we compare it with existing techniques and run testing on three different types of Bluetooth stacks covering five distinct profiles. Our experiments reveal that FuBuKi surpasses existing methods, particularly in its application to higher-level Bluetooth profiles. This paper makes the following main contributions:

- We present FuBuKi, a binary-rehosting Bluetooth fuzzer with a profile emulation kit designed for BR/EDR application profiles above L2CAP. Our approach enables Bluetooth fuzzing without requiring physical devices or target source code. In addition, FuBuKi's architecture supports parallel fuzzing operations, significantly reducing the time required to uncover vulnerabilities by running multiple instances simultaneously.
- We design a methodology to reach application profiles effectively.
- We evaluate our framework using real-world Bluetooth stacks to demonstrate its effectiveness in binary coverage, comparing it to other state-of-the-art methods.
- Using FuBuKi, we identified previously unacknowledged crashes in a commercial automotive firmware. Additionally, we employed FuBuKi to detect a known CVE in Linux BlueZ (Holtmann et al., 2024). These findings underscore FuBuKi's effectiveness in uncovering real-world vulnerabilities.

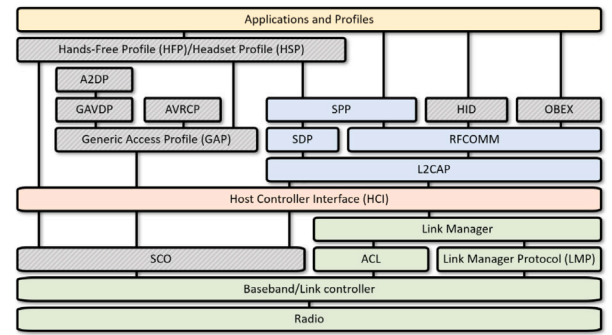


Fig. 2. Bluetooth BR/EDR architecture and its profiles (CypressAcademy, 2024).

1.4. Organization

Section 2 introduces the background knowledge pertinent to our research. Following that, Section 3 reviews related research on Bluetooth fuzzing. Section 4 outlines the specific problems we aim to address in Bluetooth fuzzing. Section 5 details our proposed methodology for solving the identified problems. Section 6 presents the results of experiments conducted on real-world Bluetooth software. Section 7 discusses the implications and limitations of our work and potential future research directions. Finally, Section 8 provides the conclusions drawn from our research.

2. Preliminaries

In this section, we will first introduce an overview of the BR/EDR Bluetooth stack and the functionalities of its profiles. We will then discuss the state-of-the-art work conducted on fuzzing with problems applied to real-world targets.

2.1. Overview of bluetooth BR/EDR and its profiles

Bluetooth BR/EDR (Basic Rate/Enhanced Data Rate) is a core Bluetooth wireless communication stack technology designed primarily for short-range, high-speed data exchange. It is widely used in various devices for audio streaming, file transfers, and other wireless communications. A typical Bluetooth system comprises two main components: the controller stack and the host stack. The controller stack is often implemented as a separate chip or module and can be upgraded through firmware updates sent from the host stack. HCI (Host Controller Interface) command requests facilitate communication between the host and controller stacks. These commands enable the host stack to control basic functions of the controller stack, such as performing hardware resets and managing advertising. This interaction forms the lowest layer of the Bluetooth protocol within the operating system.

Beyond the HCI layer, several higher-level protocols and profiles enhance Bluetooth's functionality, as shown in Fig. 2. One of these is L2CAP (Logical Link Control and Adaptation Protocol), which plays a crucial role in connection establishment and packet reassembly. Another key component in the Bluetooth stack is the Service Discovery Protocol (SDP), which allows devices to discover available services and their attributes on other Bluetooth-enabled devices. AVRCP (Audio/Video Remote Control Profile) is widely used for audio and media control applications. AVRCP provides a standardized interface to control media playback functions such as playing, pausing, and skipping on remote devices, commonly seen in wireless headphones and car audio systems. RFCOMM (Radio Frequency Communication) is a protocol that emulates serial port communication, enabling Bluetooth devices to support applications that traditionally rely on serial connections. In practice, these BR/EDR protocols are implemented by host-side Bluetooth stacks such as BlueZ (Holtmann et al., 2024) and BTstack (BTstack, 2024).

2.2. Greybox fuzzing

Greybox fuzzing is one of the most effective approaches to discovering vulnerabilities in software applications, blending the strengths of black-box and white-box testing. This method leverages partial knowledge of the program's internal structure to intelligently guide the fuzzing process while employing random or semi-random inputs to test the software. A leading tool in this domain is AFL++ (Fioraldi et al., 2020), which uses lightweight code instrumentation to monitor execution paths and feedback from code coverage to optimize the mutation of inputs. Thus, AFL++ (Fioraldi et al., 2020) can efficiently uncover many bugs and security flaws. However, AFL++ relies on LLVM compilers for instrumentation (Fioraldi et al., 2020), limiting its use to applications with available source code. To address this, the community has developed the Quick Emulator (QEMU) (Bellard, 2005) mode plugin, enabling users to fuzz binary files without needing the source code, thereby expanding AFL++'s applicability to a broader range of software.

3. Related work

In this section, we will introduce some related works.

3.1. State machine based bluetooth fuzzers with physical dongle

A recent survey by Zheng et al. (2025) provides a comprehensive taxonomy of network protocol fuzzing, highlighting that while state awareness is critical, achieving it without access to source code remains a distinct challenge. In 2020, Garbelini et al. (2020) developed a fuzzer that utilizes a predefined state machine to generate Bluetooth Low Energy (BLE) packets transmitted over the air via a physical Bluetooth dongle. This fuzzer then monitors the responses to these packets to identify any anomalies or crashes in the target system. Similarly, Park et al. (2022) created a fuzzer based on a state machine for the L2CAP protocol. Their approach includes using the coverage ratio of L2CAP states as feedback to guide and optimize the fuzzing process.

3.2. Fuzzers with specialized hardware

In a different approach, Garbelini et al. (2022), in 2022, used reverse engineering to craft and receive Link Manager Protocol (LMP) packets, effectively specializing a Bluetooth device for this purpose. Heinze et al. (2020) took a novel approach by jailbreaking an iPhone and using Frida (Frida, 2024) to inject fuzzer input directly into function memory. Although these methods can be effective, they require a physical board or dongle to transmit data to the target system, which complicates the scalability of large-scale fuzzing operations.

3.3. Emulation-based and rehosting-based bluetooth fuzzers

Conversely, Ruge et al. (2020) extracted Bluetooth chip firmware and utilized QEMU (Bellard, 2005) to emulate the controller, allowing interaction with the host system to uncover vulnerabilities in the chip and host. Garbelini et al.'s BrakTooth (Garbelini et al., 2022) further showed that controller/firmware-oriented Bluetooth testing can expose severe implementation flaws; however, the workflow still relies on specialized physical hardware. Lei et al. (2022) crafted a fuzzer with a teletype (TTY) device interface and a state-based BLE packet generator. More recently, VirtFuzz (Huster et al., 2024) demonstrated hardware-free fuzzing with virtualized controller emulation. These approaches are important, but they mainly target low-level protocols and heavily rely on VirtIO support. Also, they do not directly address binary-only host-side BR/EDR application profiles above L2CAP, where the key challenge is to preserve a valid session while mutating profile-specific payload fields.

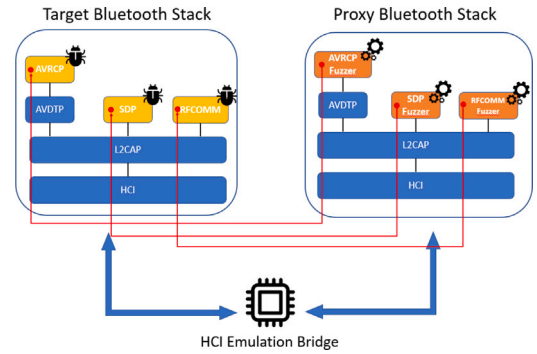


Fig. 3. FuBuKi's profile-aware proxy for reaching high-level profiles.

Positioning of FuBuKi. FuBuKi addresses a complementary technical gap. Rather than fuzzing controller firmware itself or relying on over-the-air delivery, FuBuKi rehosts the target host-side Bluetooth binary and virtualizes the controller boundary at the HCI/HAL interface. This design enables coverage-guided fuzzing of binary-only BR/EDR application profiles, such as SDP, RFCOMM, and AVRCP, without requiring physical Bluetooth hardware. For the evaluated targets, this also avoids tight coupling to an emulator-specific device model and allows the same fuzzing architecture to be applied to Linux daemons, embedded user-space stacks, and extracted vendor binaries.

4. Problem statement

In this paper, our focus will be on detecting vulnerabilities in application profiles above L2CAP. While several studies have been conducted, none have specifically targeted application profiles. The primary challenge with fuzzing high-level profiles can be categorized as follows.

4.1. Challenge 1: Reaching application profiles efficiently

Most Bluetooth application profiles are stateful, but the main difficulty is not state modeling alone; it is reaching a profile-specific parser with semantically valid packets after connection setup, channel negotiation, and sometimes pairing. Some prior studies construct state machines and alter packets according to state transitions (e.g., BT-Fuzz Lei et al., 2022, L2Fuzz Park et al., 2022). These approaches are effective for lower-layer exploration, but they do not by themselves provide a practical way to inject mutations into higher-level BR/EDR profile payloads while preserving a valid session. Our solution therefore uses a *profile-aware proxy*: by leveraging an existing Bluetooth stack as the peer, the user can establish the prerequisite session and then inject fuzzed bytes into a selected profile field, as depicted in Fig. 3.

4.2. Challenge 2: Fuzzing without physical machines

Another problem is that most Bluetooth stacks are closed-source and usually implemented as long-lived daemons, making them difficult to fuzz directly with AFL++ (Fioraldi et al., 2020) and related coverage-guided fuzzers. Heinze et al. (2020) employed Frida (Frida, 2024) to instrument the iOS Bluetooth stack and inject fuzzing inputs directly into function memory. However, this method still necessitates a jailbroken device. An alternative line of work performs over-the-air fuzzing using a physical dongle to communicate with the target device (e.g., Stateful Black-Box Fuzzing of Bluetooth Devices Using Automata Learning Pferscher and Aichernig, 2022, L2Fuzz Park et al., 2022), followed by mutating packets based on the observed responses. Nonetheless, this method lacks scalability and makes it harder to isolate the root cause of crashes.

Our methodology combines binary rehosting with AFL++ QEMU mode and an HCI emulation utility. QEMU mode is used to collect binary coverage without source code (Fioraldi et al., 2020). Separately, a one-time harness patch is used only to expose or intercept an internal target parser so that the daemon can be driven repeatedly under fuzzing. Compared with on-device instrumentation approaches such as ToothPicker (Heinze et al., 2020), our design executes the extracted host-side binary in a controlled user-space environment. Compared with over-the-air fuzzing approaches, FuBuKi eliminates radio and hardware bottlenecks and can scale using AFL++'s parallel fuzzing capability (AFLplusplus, 2024).

4.3. Challenge 3: Low or unknown overall coverage ratio

The final problem is that most approaches, such as black box fuzzers, depend on response packets (e.g., SweynTooth Garbelini et al., 2020, L2Fuzz Park et al., 2022). These fuzzers can only measure partial coverage in specific profiles according to state or model transitions, which may miss some target-specific implementation faults in real-world applications. Although Heinze et al. (2020) successfully utilize Frida Stalker (Frida, 2024) to collect binary coverage, they still cannot support high-level profiles that require pairing, such as AVRCP. As a result, they achieve only 3% of total coverage in the iOS Bluetooth stack.

Our methodology utilizes the AFL++ QEMU mode (Fioraldi et al., 2020) to collect binary coverage at runtime, providing our fuzzer with better insights into the internals of the Bluetooth stack. Additionally, our framework can reach high-level profiles that previous work could not, significantly increasing the total coverage ratio in the target Bluetooth stack.

4.4. Threat model and evaluation scope

FuBuKi targets vulnerabilities in *host-side BR/EDR profile logic* under the assumption that a peer device can communicate with the target over a valid Bluetooth session. In practice, this means the adversary or tester may first satisfy the protocol's required preconditions, such as ACL connection establishment, L2CAP channel negotiation, and, when necessary, pairing or profile activation. The focus of FuBuKi is therefore not exhaustive exploration of all lower-layer connection states, but *coverage-guided robustness testing of upper-layer profile handlers once the session prerequisites are met*.

This scope is important for interpreting our comparisons. Some prior tools primarily target lower-layer or end-to-end interaction problems and must spend substantial effort on connection establishment, whereas FuBuKi is optimized for testing the application/profile layer after the setup barrier has been crossed. To avoid conflating these goals, our evaluation separates *post-handshake application-layer fuzzing* from *pre-handshake end-to-end fuzzing*.

5. Methodology

In this section, we will introduce our methodology.

5.1. Overview of FuBuKi

FuBuKi is a coverage-guided fuzzing framework designed to assess the security of binary-only Bluetooth stacks. As depicted in Fig. 4, the architecture consists of four distinct components: (1) the *Profile-Aware Proxy*, which establishes the required session and transforms fuzzing inputs; (2) the *Target Environment*, comprising the rehosted and harness-instrumented binary; (3) the *HCI Emulation Bridge*, which virtualizes the physical controller interface; and (4) the *Fuzzing Engine* (AFL++ Fioraldi et al., 2020).

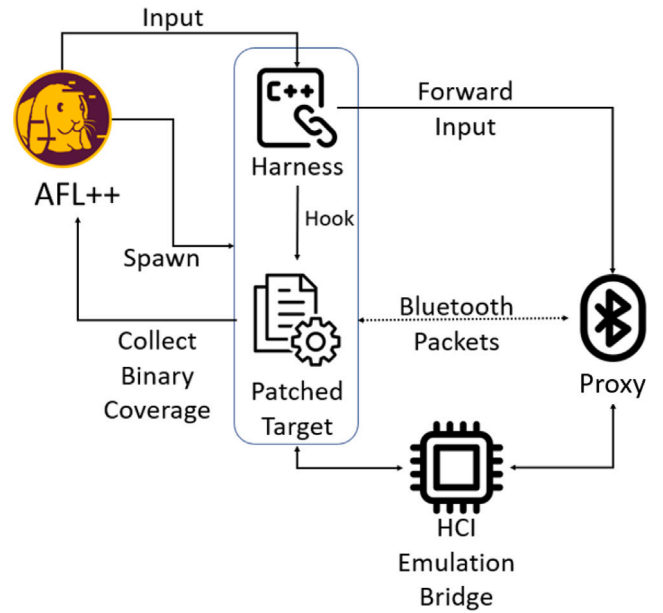


Fig. 4. The FuBuKi architecture. The system decouples the target stack from physical hardware using an HCI Emulation Bridge. The Profile-Aware Proxy establishes the prerequisite session and applies profile-specific input transformation, while AFL++ drives mutation based on binary coverage feedback.

5.2. The profile-aware proxy and input transformation

A core challenge in Bluetooth fuzzing is that random bit-flipping at the packet level rarely reaches deep profile logic; packets are dropped early if the device is disconnected, if channels are unopened, or if profile headers are malformed. To address this, FuBuKi uses a **Profile-Aware Proxy**. The proxy executes a small, user-defined session script and a profile-specific Input Transformer to keep the setup valid while exposing selected payload bytes to AFL++.

5.2.1. Role of the proxy

The Proxy runs as a separate process and acts as the peer device in the emulation environment. It is responsible for:

1. **Session Establishment:** Establishing the baseband (ACL) connection, performing pairing/authentication when needed, and negotiating the required L2CAP or higher-layer channels. This ensures that the target is in the correct state to accept application-layer data.
2. **Profile-Specific Input Transformation:** This is the integration point for the fuzzer. The user defines an Input Transformer script using standard Python Bluetooth libraries (e.g., Bumble or Scapy). The script specifies a valid packet template for the target profile, such as an SDP request, an RFCOMM frame, or an AVRCP command.

5.2.2. Workflow of transformation

Instead of generating an entire Bluetooth session from scratch, AFL++ produces a raw mutation buffer. The Input Transformer injects this mutated buffer into a designated payload field of a valid profile packet template. For example, when testing SDP, the Proxy handles the L2CAP connection, while the fuzzed bytes are inserted only into the *ServiceSearchAttributeRequest* payload. This hybrid design keeps lower-layer headers, framing, and checks valid enough to traverse the stack, while still allowing the mutated payload to exercise deep parser logic.

Table 1

Manual effort for the profile-specific Input Transformers used in our evaluation. Python LoC counts the logic that establishes a valid session and injects mutated bytes into a profile-specific payload. Auxiliary helper code, when present, is platform-specific and unrelated to Bluetooth profile semantics.

Target	Python LoC	Aux. C LoC
Custom L2CAP App	62	0
Custom RFCOMM Service	67	0
BTstack SDP Target	140	0
Commercial Automotive Firmware	43	108
BlueZ AVRCP (CVE-2023-27349)	217	0

5.2.3. Manual effort and why It is necessary

Because the Input Transformer is authored per target profile, FuBuKi requires limited but non-zero manual adaptation effort. Table 1 quantifies this effort for the five evaluated targets. The Python transformer logic ranges from 43 to 217 lines of code in our evaluation. One automotive target additionally required 108 lines of auxiliary platform-specific C helper code. Importantly, this helper logic is unrelated to Bluetooth profile semantics; it was used only to expose an internal entry point for the harness in that firmware.

This design choice differs from automated state-learning fuzzers such as AFLNet (Pham et al., 2020). AFLNet is highly effective for request-response protocols with observable message boundaries and response codes, but several Bluetooth BR/EDR profiles do not expose a stable one-request/one-response structure for every mutated message, such as A2DP RTP frame transmission or L2CAP connectionless. Some control/data paths are asynchronous, and lower-layer events do not always map cleanly to profile-level parser states. In addition, protocol-specific message delimitation is still required before state feedback can be learned. For this reason, FuBuKi does not claim fully automatic state discovery. Instead, it uses a compact, profile-specific transformer to preserve valid setup steps and focus mutations on the semantic fields that actually reach deep profile logic.

5.3. Target rehosting, harness injection, and fork-server execution

Most Bluetooth stacks run as long-lived daemons (e.g., `bluetoothd`) that do not terminate after processing a single request. As a result, standard AFL++ execution semantics cannot be applied directly, because the target would otherwise be treated as a timeout rather than a completed test case.

To fuzz such binaries without source code, FuBuKi combines two separate mechanisms: (i) AFL++ QEMU mode for runtime binary coverage collection (Fioraldi et al., 2020), and (ii) a one-time harness patch for stable function interception and re-entry. Since AFL++ persistence only takes a memory snapshot and restores it for the target binary, it often directly destroys the connection state. As a result, we use LIEF (Thomas et al., 2024) only to rehost the binary and inject a small trampoline around the selected parser entry point identified during reverse engineering. The offline patch is therefore *not* the runtime coverage backend; QEMU remains solely responsible for coverage feedback.

- 1. Target Function Identification:** Using reverse engineering, aided by debug symbols or log strings as shown in Fig. 5, we identify the function responsible for processing the target profile data (e.g., `avrcp_handle_pdu`).
- 2. Harness Injection:** We use LIEF (Thomas et al., 2024) to expose or intercept the identified entry point and inject a small trampoline harness. This harness initializes the daemon, transfers AFL++ inputs to the selected parser context, and redirects execution into the fuzzing loop in a reproducible way across runs.

```

36 if ( !v1 )
37 {
38     _sprintf_chk(&dest[31088], 1LL, 5LL, "%hu", (unsigned __int8)dest[10508]);
39     sub_1A4E40(786436LL, "btm_io_capabilities_req() State: %s", &dest[31088]);
40 }
41 v2 = sub_7A300(&v10);
42 *(_BYTE *) (v2 + 353) |= 0x11u;
43 v3 = v2;
44 if ( (unsigned __int8)dest[31020] > 4u )
45 {
46     v16 = (unsigned __int8)dest[10509];
47     v17 = *(_QWORD *)v2;
48     _sprintf_chk(&dest[31088], 1LL, 5LL, "%hu", (unsigned __int8)dest[10508]);
49     sub_1A4EC0(
50         786436LL,
51         "btm_io_capabilities_req() State: %s  Flags: 0x%04x  p_cur_service: 0x%08x",
52         &dest[31088],
53         v16,
54         v17);
55 }
56 v4 = dest[10508];
57 v5 = dest[10485];
58 if ( !v4 )

```

Fig. 5. Decompiled Bluetooth code with debug log.

- 3. Fork-Server Execution:** Once the hook point is reached, the process forks. The child process executes the target function with the fuzzed input, reports coverage to AFL++ via shared memory, and then exits. The parent process remains alive and preserves the prepared daemon state, thereby avoiding the cost of restarting the full Bluetooth stack from scratch for every test case.

This design preserves a *prepared baseline state* in the parent process, such as an already established ACL connection, negotiated L2CAP channel, or activated profile context. Each fuzz iteration then forks from that prepared state and executes one selected fuzzed message in the child. Accordingly, FuBuKi currently follows a *fixed-prefix, single-mutation* model: the proxy or harness may deterministically send setup packets before the fuzzed packet, but the current prototype does not yet perform fully general sequence fuzzing in which multiple packets within the same execution are independently mutated and jointly coverage-guided.

We chose this design because our primary requirement is to invoke or intercept internal host-side Bluetooth handlers in heterogeneous binaries while keeping the runtime fuzzing loop simple and reproducible. A fully dynamic-hooking solution based on QEMU plugins or DBI is a valid alternative, but it would require tighter coupling to a specific instrumentation backend. In contrast, the one-time patch provides a stable hook point across runs, while QEMU handles coverage collection independently. We nevertheless acknowledge that offline patching may be unsuitable for packed binaries or targets protected by strict signature or file-integrity enforcement; extending FuBuKi with a dynamic-hooking backend is part of our future work.

5.4. HCI emulation bridge: Bridging the hardware gap

To enable scalable fuzzing without physical hardware, we utilize the Google *Rootcanal* project (hchataing et al., 2024), which emulates the behavior of the Bluetooth Controller. However, a significant integration challenge exists: *Rootcanal* exposes its virtual controller interface via a TCP network socket, whereas standard Bluetooth host stacks (e.g., BlueZ, automotive stacks) act as operating system daemons configured to communicate with hardware via serial interfaces (UART/RS232) or USB pipes.

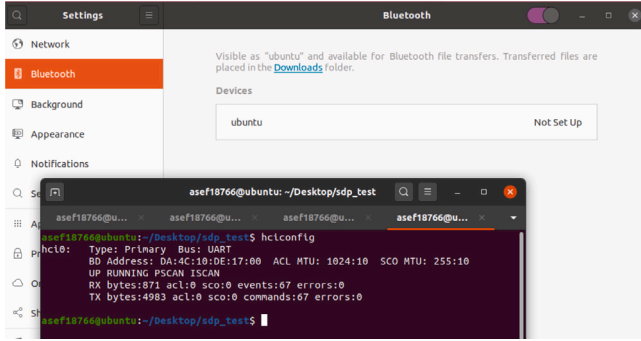
To bridge this “interface mismatch”, we developed a **Virtual Serial Shim** (as shown running in Fig. 6). This component creates a pseudo-terminal (PTY) pair on the Linux host. The target Bluetooth stack is configured to attach to this PTY as if it were a physical UART device (e.g., `/dev/ttyS0`). The shim then acts as a transducer:

1. It captures HCI H4 frames sent by the target to the PTY.
2. It encapsulates these frames into the packet format required by *Rootcanal*.
3. It forwards them over the TCP socket to the emulator, and vice-versa for incoming events.

Table 2

Time-to-First-Crash Comparison across Testing Scenarios (Limit: 24 h). ‘x’ denotes no crash found.

Target Scenario	FuBuKi (Ours)	L2Fuzz (Park et al., 2022)	Frankenstein (Ruge et al., 2020)
A. Custom L2CAP App (BlueZ)	11 s	x	x
B. Custom RFCOMM App (BlueZ)	22 min 22 s	x	x
C. Modified SDP Client (BTstack)	13 min 17 s	x	x
D. Commercial Auto Firmware (CWE-121)	9 s	x	x
E. CVE-2023-27349 (BlueZ AVRCP)	41 s	x	x

**Fig. 6.** The FuBuKi environment running the target stack (Ubuntu BlueZ) over the HCI Emulation Bridge. The stack detects a valid controller despite the absence of physical hardware.

5.5. Overall fuzzing workflow

The complete fuzzing loop operates as follows (referencing Fig. 7):

1. **Initialization:** The HCI Bridge and Emulator are started. The Target Daemon launches and attaches to the virtual UART.
2. **State Setup:** The Proxy initiates a connection to the Target. They complete the handshake (LMP/L2CAP) and enter the “Connected” state.
3. **Fuzz Loop:**
 - (a) AFL++ generates a mutation.
 - (b) The Harness (inside the Target) signals readiness.
 - (c) The Proxy receives the mutation, wraps it in the valid protocol template, and sends it over the virtual link.
 - (d) The Target receives the packet. The Harness intercepts the execution flow at the target function.
 - (e) Execution occurs under QEMU instrumentation; coverage is updated.
 - (f) The specific child process terminates; the loop repeats.

6. Evaluation

In this section, we evaluate FuBuKi to demonstrate its practical effectiveness and efficiency. We aim to answer the following research questions:

- RQ1 (Emulation Overhead):** Does the HCI Emulation Bridge introduce significant latency compared to physical Bluetooth hardware?
- RQ2 (Vulnerability Discovery):** Can FuBuKi detect real-world vulnerabilities in diverse Bluetooth stacks?
- RQ3 (Comparative Analysis):** Under fair post-handshake and pre-handshake settings, how does FuBuKi compare with representative state-of-the-art Bluetooth fuzzers over 24-hour campaigns?
- RQ4 (Parallel Scalability):** To what extent does FuBuKi benefit from multi-instance parallel fuzzing on a single workstation?

6.1. Experimental setup

All experiments were conducted on a workstation equipped with an AMD Ryzen 9 7900X CPU and 64 GB of RAM running Ubuntu 22.04. For physical hardware comparisons, we utilized a Raspberry Pi 4 Model B equipped with a TP-Link UB500 Bluetooth 5.0 dongle.

To ensure a comprehensive evaluation, we designed five distinct testing scenarios that cover different layers of the Bluetooth stack and multiple implementation styles.

1. **Scenario A (BlueZ L2CAP):** A custom server application built on the Linux BlueZ stack listening on an L2CAP Protocol/Service Multiplexer (PSM).
2. **Scenario B (BlueZ RFCOMM):** A legacy serial port emulation service running on BlueZ, requiring valid RFCOMM channel negotiation.
3. **Scenario C (BTstack SDP):** A modified Service Discovery Protocol (SDP) client based on the embedded *BTstack* library (BTstack, 2024), configured to parse complex service records.
4. **Scenario D (Automotive ECU):** A proprietary, closed-source Bluetooth stack extracted from a commercial automotive firmware, running a simplified Media Control service.
5. **Scenario E (BlueZ AVRCP):** The standard audio control profile implementation in Linux BlueZ, specifically targeting the metadata parsing logic associated with CVE-2023-27349.

Moreover, to distinguish profile-layer effectiveness from end-to-end connection behavior, we separate the comparative experiments into two settings. In the **post-handshake** setting, both tools start from an equivalent prepared application-layer state so that the comparison isolates profile-layer fuzzing effectiveness after connection establishment. In the **pre-handshake** setting, both tools start before connection setup, and the connection process remains part of the tested behavior. We use 24-hour campaigns for the main comparative coverage experiments, following standard guidance for fuzzing evaluation methodology (Klees et al., 2018).

In RQ1 to RQ3, all experiments use a single fuzzing instance. We reserve multi-instance execution exclusively for RQ4, whose purpose is to evaluate FuBuKi’s parallel scalability. This separation avoids confounding factors such as CPU contention and resource interference in the latency, vulnerability-discovery, and comparative-coverage experiments.

6.2. RQ1: Performance overhead analysis

To verify the viability of our emulation approach, we compared the packet transport speed of FuBuKi against a physical Bluetooth setup. We executed a benchmark involving 1000 sequential SDP service record requests and responses. The Service Discovery Profile was selected because it runs over L2CAP and does not require encryption, allowing us to measure raw transport latency without cryptographic overhead.

As shown in Fig. 8, the physical Bluetooth device processed requests with an average latency of **0.08 s**. In contrast, the FuBuKi emulation environment averaged **0.23 s** per request—approximately 3× slower. **Analysis of Overhead:** The observed latency in FuBuKi is primarily attributed to the architecture of the HCI Emulation Bridge described in Section 5.4. In a physical setup, the kernel moves data directly from the USB driver to the socket buffer. In FuBuKi, a packet must traverse:

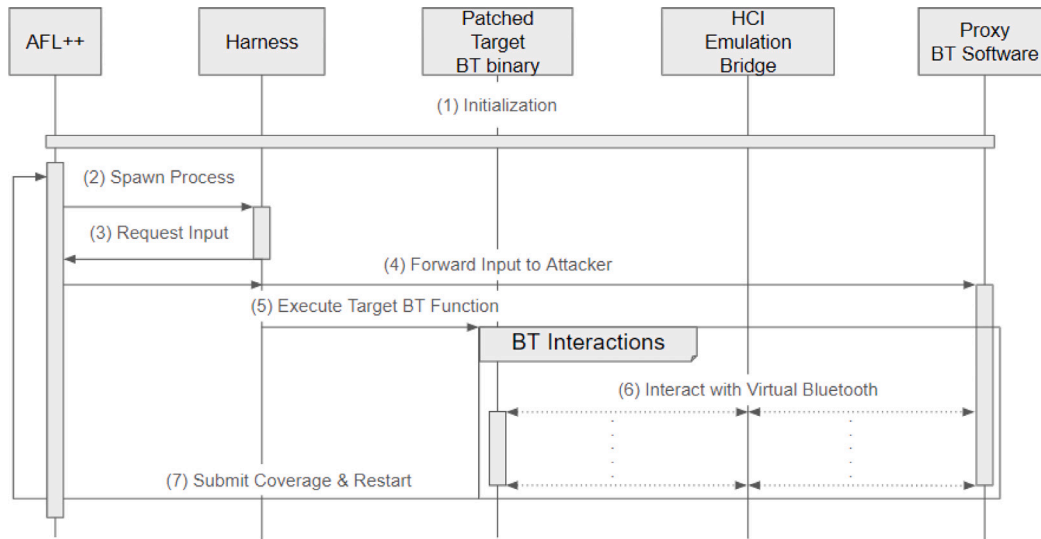


Fig. 7. FuBuKi workflow.

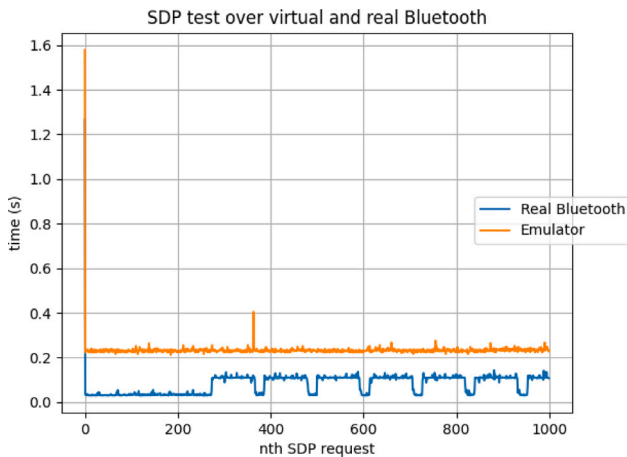


Fig. 8. Latency comparison for 1000 SDP requests. The physical device (blue) maintains a lower average latency (≈ 0.08 s) compared to the emulator (orange, ≈ 0.23 s), but the emulator allows for massive parallelization. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

1. The Target's User-Space $\rightarrow$ Kernel (PTY Write)
2. Kernel $\rightarrow$ Virtual Serial Shim (Context Switch)
3. Shim $\rightarrow$ Rootcanal (TCP Socket Write)
4. Rootcanal Processing $\rightarrow$ Shim (TCP Socket Read)
5. Shim $\rightarrow$ Kernel (PTY Write)
6. Kernel $\rightarrow$ Target (Context Switch)

This introduces multiple context switches per packet. However, this latency is an acceptable trade-off. Unlike physical fuzzing, which is limited by the number of available USB dongles and radio interference, FuBuKi instances are isolated virtual processes. We can run dozens of instances in parallel on a single CPU, achieving a cumulative throughput that vastly exceeds the physical hardware's capabilities, despite the per-packet latency.

6.3. RQ2: Vulnerability discovery capabilities

To answer RQ2, we subjected the five target scenarios described in the Experimental Setup to a comparative fuzzing campaign. We tested FuBuKi against two state-of-the-art baselines: L2Fuzz (Park et al., 2022)

and Frankenstein (Ruge et al., 2020). Each tool was given a budget of one hour per target.

Table 2 summarizes the time-to-first-crash for each tool. The results indicate a distinct advantage for FuBuKi in high-level application profiles.

6.3.1. Analysis of baseline failures

The baseline tools failed to trigger crashes in the application-layer scenarios (B, C, D, E).

- **L2Fuzz** relies on a learned state machine primarily optimized for L2CAP. In tests requiring complex upper-layer handshakes (e.g., establishing an RFCOMM channel or an AVRCP browsing channel), L2Fuzz repeatedly stalled in its internal state machine, unable to negotiate the prerequisites to reach the vulnerable code.
- **Frankenstein**, while capable of emulating firmware, struggled to generate valid HCI event sequences required to satisfy the host stack's strict state checks. It successfully injected packets, but the host stack dropped them due to invalid connection handles or sequence numbers.

6.3.2. Case study 1: Commercial automotive firmware (scenario D)

One of the most significant findings was a vulnerability in a closed-source Bluetooth stack extracted from a famous electronic automotive ECU. The stack implements a legacy media control service.

FuBuKi identified a crash in just 9 s. Post-mortem analysis of the core dump revealed a Stack-based Buffer Overflow (CWE-121). The vulnerability existed in the function responsible for parsing vendor-specific media attributes. This finding aligns with recent insights by Rahimi et al. (2025), who highlight that heavy-duty and commercial vehicular systems often lack the robust cybersecurity scrutiny applied to consumer electronics, leaving critical intra-vehicle networks vulnerable to exposure.

6.3.3. Case study 2: Rediscovering CVE-2023-27349 (scenario E)

To validate FuBuKi's ability to find known critical bugs, we targeted the AVRCP profile in an older version of Linux BlueZ. CVE-2023-27349 (NVD, 2024) is a vulnerability where handling a specific, malformed Protocol Data Unit (PDU) triggers a crash.

FuBuKi triggered this vulnerability in 41 s. Fig. 9 presents the packet capture of the exploit. The capture demonstrates the efficacy of our "Input Transformer" methodology (Section 5.2). The outer layers (HCI, L2CAP, AVCTP) are structurally valid, allowing the packet to pass through the lower stack layers. However, inside the AVCTP payload,

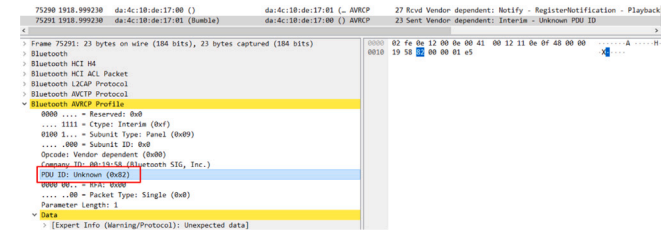


Fig. 9. Wireshark capture of the crash-triggering packet generated by FuBuKi for CVE-2023-27349. The fuzzer successfully mutated the AVRCP PDU ID to the invalid value 0x82 (Unknown) while maintaining a valid AVCTP/L2CAP wrapper.

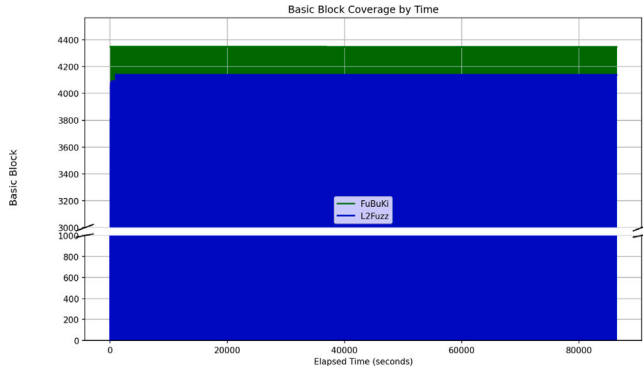


Fig. 10. 24-hour post-handshake basic-block coverage on the BTstack A2DP source server. Both FuBuKi and L2Fuzz start from an equivalent prepared session state. FuBuKi maintains a consistent coverage advantage throughout the campaign.

FuBuKi mutated the **PDU ID** field to 0x82 (highlighted in blue). The target function ‘avrcp_handle_pdu’ did not have a default case for this unknown ID in its switch statement, leading to an assertion failure and daemon termination. This confirms that FuBuKi can explore “unseen” branches in stateful protocols by mutating specific payload fields while keeping the session alive.

6.4. RQ3: Comparative analysis

To provide a clearer comparison across tools with different operating assumptions, we report two complementary 24-hour evaluation settings. The first isolates application-layer fuzzing after session establishment, while the second measures end-to-end behavior from the pre-handshake state.

6.4.1. Setting A: Post-handshake application-layer comparison against L2Fuzz

Our first comparison isolates application-layer fuzzing effectiveness after session establishment. We configured both FuBuKi and L2Fuzz (Park et al., 2022) to start from an equivalent post-handshake state on the same BTstack A2DP source-server target. This prevents either tool from being penalized or advantaged by connection-establishment overhead and directly measures their ability to explore profile-level logic once the session barrier has been crossed.

Fig. 10 shows that under this fair post-handshake setting, FuBuKi still achieves higher cumulative basic-block coverage than L2Fuzz over the full 24-hour horizon. This result indicates that FuBuKi’s advantage is not solely an artifact of starting from a prepared session. Instead, the profile-aware input transformation continues to provide better reach into the target’s higher-level parsing logic even when both tools are given the same application-layer starting point.

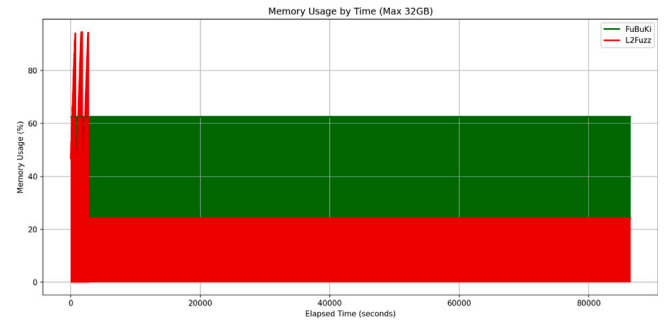


Fig. 11. 24-hour memory usage comparison (32 GB host) during the post-handshake comparison. FuBuKi remains stable, while L2Fuzz exhibits repeated memory spikes and unstable behavior.

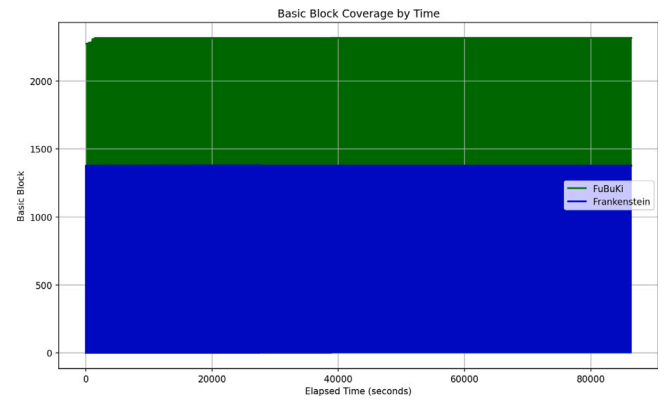


Fig. 12. 24-hour pre-handshake basic-block coverage comparison against Frankenstein. Both tools start before handshake. FuBuKi reaches deeper protocol logic and maintains substantially higher coverage throughout the campaign.

We also monitored memory behavior during this experiment. As shown in Fig. 11, FuBuKi maintained stable memory consumption, whereas L2Fuzz exhibited repeated high-memory excursions during the campaign. This observation is consistent with the fact that eliminating hardware-dependent reconnection loops and reusing a prepared execution structure improves not only coverage growth but also operational stability.

6.4.2. Setting B: Pre-handshake end-to-end comparison against Frankenstein

Our second comparison evaluates the end-to-end setting in which connection establishment remains part of the challenge. We compared FuBuKi against Frankenstein Ruge et al. (2020) on a BTstack SDP target that is exposed before pairing, so both tools begin before handshake and interact with the target from the same initial state.

As shown in Fig. 12, FuBuKi achieves substantially higher coverage than Frankenstein over the entire 24-hour campaign. The difference emerges early and persists throughout the run, suggesting that FuBuKi’s HCI emulation bridge and host-side profile reachability remain effective even when the session establishment cost is included in the budget.

6.4.3. Interpretation

Taken together, these two settings provide a fairer and more informative picture than our original mixed comparison. Under the post-handshake setting, FuBuKi still outperforms L2Fuzz, showing that the advantage is not reducible to an easier starting state. Under the pre-handshake setting, FuBuKi also outperforms Frankenstein in an end-to-end campaign, demonstrating that the framework remains effective when session establishment is part of the tested behavior.

Table 3
Parallel scaling behavior of FuBuKi on the BTstack SDP client.

Instances	Avg. Memory	Exec/s per Inst.	Total Exec/s
5	1.76 GB	0.43	2.15
10	2.03 GB	0.42	4.20
15	3.40 GB	0.44	6.60
20	3.62 GB	0.42	8.40
25	3.54 GB	0.37	9.25
30	3.89 GB	0.34	10.20
35	4.00 GB	0.34	11.90
40	4.05 GB	0.34	13.60
45	4.10 GB	0.33	14.85
50	4.28 GB	0.33	16.50
55	4.30 GB	0.31	17.05
60	CPU overload	–	–

These results support the main design claim of FuBuKi: once host-side BR/EDR profile logic is made reachable through binary rehosting and HCI virtualization, coverage-guided fuzzing can explore substantially deeper application-layer code than lower-layer- or firmware-oriented baselines (Park et al., 2022; Ruge et al., 2020; Huster et al., 2024).

For completeness, we provide the corresponding 1-hour coverage curves in Appendix. These short-horizon results are included only to illustrate early-stage warm-up behavior; our main comparative claims are based on the 24-hour campaigns in Section 6.4.

6.5. RQ4: Parallel scalability

A central design goal of FuBuKi is to eliminate the hardware bottleneck that constrains conventional Bluetooth fuzzing. To assess this property, we evaluated the scalability of FuBuKi on the **BTstack SDP client** target by increasing the number of concurrent instances running on a single workstation. For each configuration, we measured the average memory consumption, the per-instance execution throughput, and the total aggregate throughput.

As shown in Table 3, FuBuKi exhibits strong parallel scaling on a single host. The aggregate throughput rises from **2.15 exec/s** at 5 instances to **17.05 exec/s** at 55 instances, yielding an approximately **7.9×** increase in total throughput. While the per-instance execution rate remains stable up to 20 instances, it gradually declines thereafter, dropping from roughly **0.42–0.44 exec/s** to **0.31 exec/s** at 55 instances. This behavior is expected and indicates increasing contention for shared CPU resources.

Importantly, memory consumption grows much more slowly than the instance count. Even at 55 concurrent instances, the average memory usage is only **4.30 GB**, suggesting that memory is not the primary scaling constraint in our setup. Instead, the limiting factor is CPU saturation: at 60 instances, the workstation became overloaded and could no longer sustain stable fuzzing execution.

Overall, these results confirm that FuBuKi’s main performance advantage lies in *aggregate parallel throughput* rather than single-instance speed. By replacing physical Bluetooth hardware with a virtualized HCI bridge and rehosted target environment, FuBuKi can scale to dozens of simultaneous fuzzing workers on commodity hardware. This directly validates the framework’s hardware-independent parallelization claim.

7. Discussion and future work

In this section, we discuss the implications of our findings, address the trade-offs inherent in our design, and outline future directions.

7.1. Practical benefits of binary rehosting

A practical advantage of FuBuKi is that it rehosts the target Bluetooth stack in a controlled software environment rather than fuzzing it on the original device. This design has several concrete benefits.

First, binary rehosting removes the dependency on physical Bluetooth hardware. Conventional Bluetooth fuzzing often requires dedicated adapters, specialized boards, or over-the-air interaction, which complicates experiment setup and limits the number of concurrent fuzzing workers that can be deployed. By virtualizing the controller boundary through the HCI emulation bridge, FuBuKi turns Bluetooth fuzzing into a software-only workload that can be reproduced on standard workstations.

Second, rehosting improves observability and reproducibility. Because the target executes as a user-space process in a controlled environment, crashes, logs, and coverage data can be collected more consistently than in device-centric settings. This is especially valuable for long-running fuzzing campaigns and post-mortem analysis, where deterministic replay and stable instrumentation are important.

Third, rehosting avoids the deployment barriers associated with on-device instrumentation. Prior approaches based on dynamic binary instrumentation on physical devices may require privileged access or device modification, such as jailbreaking, to inject hooks into system Bluetooth services (Heinze et al., 2020; Frida, 2024). In contrast, FuBuKi operates on an extracted target binary under emulation, which makes the workflow more accessible for host-side Bluetooth stacks and proprietary user-space targets.

Finally, binary rehosting aligns well with the scope of this work: fuzzing host-side BR/EDR application profiles. For such targets, the main challenge is often not controller-firmware manipulation itself, but reliably reaching profile handlers after connection setup, channel negotiation, and profile activation. Rehosting, together with the profile-aware proxy, makes these higher-layer execution paths accessible to coverage-guided fuzzing without relying on radio conditions or repeated interaction with physical devices.

Overall, binary rehosting is not merely a convenience feature in FuBuKi; it is a key enabler for scalable, observable, and repeatable fuzzing of binary-only Bluetooth host stacks.

7.2. Estimated reachability of the targeted BR/EDR logic

Using a static analysis tool, angr (Shoshitaishvili et al., 2016), we estimated the maximum theoretical coverage of the A2DP binary used in Section 6.4. We extracted all symbols and manually categorized them by profile. Since the binary includes both BR/EDR and BLE implementations, and FuBuKi currently targets BR/EDR, we filtered out BLE-specific blocks. Based on this filtered set, FuBuKi explored approximately **80.6%** of the reachable BR/EDR basic blocks in the A2DP subsystem. This confirms that once the “stateful barrier” is breached by our proxy, the fuzzer can explore the majority of the profile logic.

7.3. The trade-off of profile-aware fuzzing

A key trade-off of FuBuKi is that the Profile-Aware Proxy requires a small, profile-specific Input Transformer. This is a deliberate design choice: limited manual setup in exchange for deeper reach into application-layer logic. Automated approaches such as L2Fuzz (Park et al., 2022) avoid this per-profile scripting, but as shown in Table 2, they do not consistently reach complex upper-layer parsers. FuBuKi instead uses a compact, explicit transformer to establish a valid session and expose the semantic bytes that matter to the target parser. The implementation effort is summarized in Table 1.

To reduce this manual effort further, we plan to investigate whether large language models (LLMs) can assist in generating profile-specific transformer skeletons from reverse-engineering artifacts such as logs, symbols, and packet traces. However, as noted in our recent studies on LLM-assisted reverse engineering (Lin et al., 2025), such assistance must be paired with validation steps to prevent hallucinated protocol logic or incorrect field mappings.

7.4. Emulation vs. On-device instrumentation (the “Jailbreak” issue)

Previous works such as ToothPicker (Heinze et al., 2020) utilize dynamic binary instrumentation tools such as Frida (Frida, 2024) on physical devices. This often requires a jailbroken device to inject code into privileged Bluetooth daemons, which substantially limits the accessibility and scalability of the approach.

While FuBuKi also relies on binary modification, there is a critical distinction: **FuBuKi rehosts the extracted target binary in a controlled virtual environment.** By extracting the Bluetooth daemon or user-space stack and running it under emulation on a standard Linux workstation, we gain control over the execution environment without needing to defeat the runtime security policy of the original device. This binary-rehosting approach makes FuBuKi substantially more scalable and accessible than on-device fuzzing for the class of targets considered in this paper.

7.5. Static harness injection vs. Dynamic hooking

Our current prototype uses LIEF (Thomas et al., 2024) for one-time harness injection because it provides a stable and reproducible way to expose internal parser entry points across heterogeneous binaries. This is distinct from coverage collection, which is still handled by AFL++ QEMU mode (Fioraldi et al., 2020). We acknowledge, however, that offline patching may be unsuitable for packed binaries or targets protected by strict signature or file-integrity enforcement. In such settings, a dynamic-hooking backend based on QEMU plugins or DBI would be preferable. Extending FuBuKi with such a backend is part of our future work.

7.6. Sequence fuzzing limitation and future direction

FuBuKi currently operates in a fixed-prefix, single-mutation model. The proxy or harness may deterministically send multiple setup packets before the selected fuzzed message, which is sufficient for many host-side profile parsers that are only reachable after connection setup, channel negotiation, or profile activation. However, the current prototype does not yet perform fully general sequence fuzzing in which multiple packets in the same interaction are independently mutated and jointly optimized by coverage feedback.

This limitation is a direct consequence of our fork-server-style design. The parent preserves a prepared baseline state, and each child executes one selected fuzzed message from that state. As a result, FuBuKi is well suited for testing “state-conditioned parser bugs” but is not yet a complete solution for arbitrary long-horizon protocol-state exploration. Extending FuBuKi toward sequence-aware snapshot or stateful multi-message fuzzing is an important direction for future work.

7.7. Applicability to other platforms

The methodology of bridging a virtual TTY to a network socket is theoretically portable. However, challenges remain for operating systems that do not expose a UART interface for the Bluetooth stack. For instance, Windows utilizes a complex driver stack (mhopskins-msft, 2024) that may require a kernel-level virtual driver rather than a simple TTY shim. Similarly, fuzzing IoT firmware (e.g., ESP32) would require modifying the peripheral emulation in QEMU (QEMU, 2024) to intercept UART memory-mapped I/O (MMIO), a feature we plan to implement in future releases.

7.8. Performance optimization

While FuBuKi outperforms physical fuzzing, the context-switching overhead described in Section 6.2 remains a bottleneck. Furthermore, the need to re-establish a Bluetooth connection for every fuzzing it-

eration adds latency (0.5 to 1.4 s per handshake). Future iterations of FuBuKi will integrate snapshot-based fuzzing techniques (e.g., Nyx Schumilo et al., 2021). By taking a snapshot of the process *after* the connection is established and restoring it for each mutation, we can eliminate the handshake overhead entirely, potentially increasing throughput by an order of magnitude.

8. Conclusion

This paper presented FuBuKi, a greybox fuzzing framework for uncovering vulnerabilities in binary-only Bluetooth BR/EDR host stacks. By combining binary rehosting, an HCI emulation bridge, and a profile-aware proxy, FuBuKi enables coverage-guided fuzzing of host-side application profiles without requiring physical Bluetooth hardware or target source code.

We evaluated FuBuKi on multiple Bluetooth stacks and profiles, including Linux BlueZ, BTstack, and a proprietary automotive firmware. The results show that FuBuKi can reach deep profile logic, rediscover CVE-2023-27349, and expose a previously unreported crash in a commercial automotive target. Our long-horizon experiments further indicate that FuBuKi achieves higher coverage than representative baselines under both post-handshake and pre-handshake evaluation settings. In addition, the scaling results show that FuBuKi benefits substantially from multi-instance parallel fuzzing on commodity multi-core hardware.

Overall, these findings suggest that host-side Bluetooth profile fuzzing through binary rehosting and controller emulation is a practical and effective direction for security testing. FuBuKi currently targets fixed-prefix, single-mutation executions from a prepared session state, which is sufficient for many profile handlers that are reachable only after connection setup and profile negotiation. Extending this approach toward more general sequence-aware fuzzing remains an important direction for future work.

CRediT authorship contribution statement

Zhao Min Chen: Visualization, Software, Methodology, Investigation, Data curation, Conceptualization. **Tien-Chih Lin:** Software, Methodology, Investigation, Data curation, Conceptualization. **Guan-Yan Yang:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Investigation, Formal analysis, Conceptualization. **Yu-Sheng Lin:** Validation, Software, Methodology, Investigation, Conceptualization. **Farn Wang:** Writing – review & editing, Validation, Supervision, Project administration, Funding acquisition. **Kuo-Hui Yeh:** Writing – review & editing, Supervision, Resources, Project administration, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Kuo-Hui Yeh reports financial support was provided by National Science and Technology Council. Farn Wang reports financial support was provided by National Science and Technology Council. Guan-Yan Yang reports financial support was provided by National Science and Technology Council. Zhao Min Chen reports financial support was provided by CyCraft Technology. Tien-Chih Lin reports financial support was provided by CyCraft Technology. Yu-Sheng Lin reports financial support was provided by CyCraft Technology. Farn Wang reports financial support was provided by National Taiwan University. Farn Wang reports financial support was provided by Green Energy & Environment Research Laboratories, Industrial Technology Research Institute, Taiwan. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partially supported by the Taiwan Academic Cybersecurity Center at the National Taiwan University of Science and Technology and by the National Science and Technology Council (NSTC) under Grants 114-2221-E-002-217, 114-2622-E-A49-022, 114-2221-E-A49-210, 114-2634-F-011-002-MBK, and 114-2923-E-194-001-MY3. Additional financial support was provided by National Taiwan University (NTU) and the NTU Core Consortium Project as part of the Higher Education Sprout Project by the Ministry of Education in Taiwan, under Grants NTU-CC-114L895501, NTU-CC-115L894201 and NTU-G0647. Additional partial financial support was provided by the Department of Industrial Technology, Ministry of Economic Affairs, under the “2025 ITRI Advanced Research Program” (Grant No.: 114-EC-17-A-21-0337). Further partial support was also granted by the Hon Hai Research Institute in Taipei, Taiwan (Project No.: 114UA90042), and the Industry-Academia Innovation School, NYCU, Taiwan (Project No.: 113UC2N006). The authors would like to express their gratitude for the financial support. Moreover, thanks to Research Director Chung-Kuan Chen at CyCraft Technology for his invaluable comments. Guan-Yan Yang is grateful to the National Science and Technology Council (NSTC) in Taiwan for the graduate research fellowship (NSTC-GRF) and to Professor Shi-Cho Cha at National Taiwan University of Science and Technology for help with his research project.

Appendix. One-hour coverage curves

For completeness, we include a representative 1-hour coverage curve in this appendix. Short-horizon results are useful for visualizing early-stage warm-up behavior, such as how quickly each fuzzer reaches its initial plateau. However, because short campaigns can overemphasize setup overheads and transient effects, the main comparative conclusions in this paper are based on the 24-hour experiments reported in Section 6.4. Here, only Frankenstein is evaluated using pre-handshake basic-block coverage, whereas FuBuKi and L2Fuzz are evaluated using post-handshake basic-block coverage (see Fig. 13).

Data availability

The data that has been used is confidential.

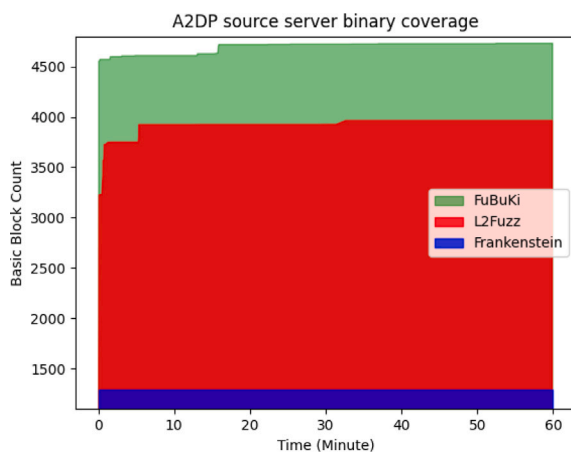
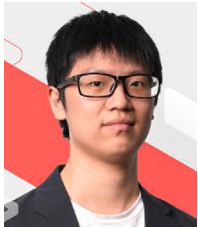


Fig. 13. Representative 1-hour cumulative coverage curve. This figure is included only to illustrate early-stage behavior; the main comparative conclusions of the paper are based on the 24-hour experiments in Section 6.4.

References

- AFLplusplus, 2024. Parallel fuzzing | AFLplusplus — aflplusplus. https://aflplusplus.com/docs/parallel_fuzzing/. (Accessed 10 June 2024).
- Bellard, Fabrice, 2005. QEMU, a fast and portable dynamic translator. In: Proceedings of the Annual Conference on USENIX Annual Technical Conference. ATEC '05, USENIX Association, USA, p. 41.
- BleedingTooth, 2024. BleedingTooth: Linux bluetooth zero-click remote code execution — google.github.io. <https://google.github.io/security-research/pocs/linux/bleedingtooth/writeup>. (Accessed 09 June 2024).
- BTstack, 2024. bluekitchen/btstack.
- Childs, Dustin, 2024. Zero day initiative — Pwn2Own vancouver 2023 - day three results — zerodayinitiative.com. <https://www.zerodayinitiative.com/blog/2023/3/24/pwn2own-vancouver-2023-day>. (Accessed 09 June 2024).
- CVE, 2024a. CVE - search results — cve.mitre.org. <https://cve.mitre.org/cgi-bin/cvekey.cgi?keyword=bluez>. (Accessed 10 June 2024).
- CVE, 2024b. CVE - search results — cve.mitre.org. <https://cve.mitre.org/cgi-bin/cvekey.cgi?keyword=linux+bluetooth>. (Accessed 10 June 2024).
- CypressAcademy, 2024. CypressAcademy_BT101_Files/PDFs/WBT101-06A-ClassicBluetooth-Basic.pdf at master · Infineon/CypressAcademy_BT101_Files — github.com. https://github.com/Infineon/CypressAcademy_BT101_Files/blob/master/PDFs/WBT101-06A-ClassicBluetooth-Basic.pdf. (Accessed 10 June 2024).
- Fioraldi, Andrea, Maier, Dominik, Eißfeldt, Heiko, Heuse, Marc, 2020. AFL++: combining incremental steps of fuzzing research. In: Proceedings of the 14th USENIX Conference on Offensive Technologies. WOOT '20, USENIX Association, USA.
- Frida, 2024. GitHub - frida/frida: Clone this repo to build Frida — github.com. <https://github.com/frida/frida>. (Accessed 10 June 2024).
- Garbelini, Matheus E., Bedi, Vaibhav, Chattopadhyay, Sudipta, Sun, Sumei, Kurniawan, Ernest, 2022. BrakTooth: Causing havoc on bluetooth link manager via directed fuzzing. In: 31st USENIX Security Symposium (USENIX Security 22). USENIX Association, Boston, MA, pp. 1025–1042.
- Garbelini, Matheus E., Wang, Chundong, Chattopadhyay, Sudipta, Sumei, Sun, Kurniawan, Ernest, 2020. SweenTooth: Unleashing mayhem over bluetooth low energy. In: 2020 USENIX Annual Technical Conference (USENIX ATC 20). USENIX Association, pp. 911–925.
- hchataing, Duarte, David, myles-android, Arya, Rahul, zxxzwu, schilit, Ferris, Christopher, Li, Xin, Girardier, Thomas, bidsharmag, Wailes, Chris, Abel, Lucas, desprezj, Larbanet, Adrien, bo hu, Slvr, dvdl, gitkaiser, Escande, William, Pandit, Abhishek, Roos, Adrian, Boccon-Gibod, Gilles, Pierce, Marshall, 2024. google/rootcanal.
- Heinze, Dennis, Classen, Jiska, Hollick, Matthias, 2020. ToothPicker: Apple picking in the iOS bluetooth stack. In: 14th USENIX Workshop on Offensive Technologies (WOOT 20). USENIX Association.
- Holtmann, Marcel, von Dentz, Luiz Augusto, Emeltchenko, Andrei, tyszczak, Gomes, Vinicius, Grzegorz, inga-s, MarcinKraglak, Uguray, Arman, Janc, Szymon, Izabela48, kumar Veeramally, Ravi, Gix, Brian, Carot-Nemesio, Santiago, gowthamab, An, Tedd Ho-Jeong, hadess, eramoto, Virtanen, Pauli, Santos, Jose, apusaka, Ikslawek, iulia-tanasescu, Sidhardhan, Syam, Lowas-Rzechonek, Michał, De Marchi, Lucas, Michalski, Rafal, bharatpanda, Szatkowski, Bartek, Kaczmarek, Andrzej, 2024. bluez/bluez.
- Huster, S., Hollick, M., Classen, J., 2024. To boldly go where no fuzzer has gone before: Finding bugs in Linux' wireless stacks through virtio devices. In: 2024 IEEE Symposium on Security and Privacy. SP, IEEE Computer Society, Los Alamitos, CA, USA, p. 28. <http://dx.doi.org/10.1109/SP54263.2024.00024>.
- Klees, George, Ruef, Andrew, Cooper, Benji, Wei, Shiyi, Hicks, Michael, 2018. Evaluating fuzz testing. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. CCS '18, Association for Computing Machinery, New York, NY, USA, pp. 2123–2138. <http://dx.doi.org/10.1145/3243734.3243804>.
- Lei, Jin, Wang, Yongjun, Zhou, Xu, Yan, Ke, 2022. BTFuzz: Accurately fuzzing bluetooth host with a simulated non-compliant controller. In: 2022 IEEE 4th International Conference on Civil Aviation Safety and Information Technology. ICCASIT, pp. 1195–1201. <http://dx.doi.org/10.1109/ICCASIT55263.2022.9987150>.
- Lin, Tien-Chih, Chao, Wei Chieh, Chen, Zhao-Min, 2025. Pay attention to the clue: Clue-driven reverse engineering by LLM in real-world malware analysis. Black Hat USA.
- mhoppkins-msft, 2024. Introduction to bluetooth profile drivers - windows drivers — learn.microsoft.com. <https://learn.microsoft.com/en-us/windows-hardware/drivers/bluetooth/bluetooth-profile-drivers-overview>. (Accessed 09 July 2024).
- NVD, 2024. NVD - CVE-2023-27349 — nvd.nist.gov. <https://nvd.nist.gov/vuln/detail/CVE-2023-27349>. (Accessed 07 June 2024).
- Park, Haram, Nkuba, Carlos Kayembe, Woo, Seunghoon, Lee, Heejo, 2022. L2Fuzz: Discovering bluetooth L2CAP vulnerabilities using stateful fuzz testing. In: 2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks. DSN, IEEE, <http://dx.doi.org/10.1109/dsn53405.2022.00043>.
- Pferscher, Andrea, Aichernig, Bernhard K., 2022. Stateful black-box fuzzing of bluetooth devices using automata learning. In: Deshmukh, Jyotirmoy V., Havelund, Klaus, Perez, Ivan (Eds.), NASA Formal Methods. Springer International Publishing, Cham, pp. 373–392.

- Pham, Van-Thuan, Böhme, Marcel, Roychoudhury, Abhik, 2020. AFLNET: A greybox fuzzer for network protocols. In: 2020 IEEE 13th International Conference on Software Testing, Validation and Verification. ICST, pp. 460–465. <http://dx.doi.org/10.1109/ICST46399.2020.00062>.
- QEMU, 2024. esp-toolchain-docs/qemu at main · espressif/esp-toolchain-docs — github.com. <https://github.com/espressif/esp-toolchain-docs/tree/main/qemu>. (Accessed 09 June 2024).
- Rahimi, Narges, Schuelke-Leech, Beth-Anne, Mirhassani, Mitra, 2025. A comprehensive review of security vulnerabilities in heavy-duty vehicles: Comparative insights and current research gaps. *Comput. Secur.* 154, 104452. <http://dx.doi.org/10.1016/j.cose.2025.104452>.
- Ruge, Jan, Classen, Jiska, Gringoli, Francesco, Hollick, Matthias, 2020. Frankenstein: Advanced wireless fuzzing to exploit new bluetooth escalation targets. In: 29th USENIX Security Symposium (USENIX Security 20). USENIX Association, pp. 19–36.
- Schumilo, Sergej, Aschermann, Cornelius, Abbasi, Ali, Wörner, Simon, Holz, Thorsten, 2021. Nyx: Greybox hypervisor fuzzing using fast snapshots and affine types. In: 30th USENIX Security Symposium (USENIX Security 21). USENIX Association, pp. 2597–2614.
- Seri, Ben, Vishnepolsky, Gregory, 2017. BlueBorne. 888.
- Shoshitaishvili, Yan, Wang, Ruoyu, Salls, Christopher, Stephens, Nick, Polino, Mario, Dutcher, Audrey, Grosen, John, Feng, Siji, Hauser, Christophe, Kruegel, Christopher, Vigna, Giovanni, 2016. SoK: (state of) the art of war: Offensive techniques in binary analysis. In: IEEE Symposium on Security and Privacy.
- Thomas, Romain, tsunekoh, aguinetqb, Sanders, David, ACSC-CyberLab, Matt, Montoya, Antonio Flores, fanquake, demons, mgunyo, Donnelly, Ray, Kilmer, Eric, Brunner, Louis, yd0b0N, pbrunet, Williams, Andrew, Duarte, Marcelo, serge-sans-paille-qb, Santiago, Rafael, liyansong2018, liumquing, junghee, mackncheesiest, Zhang, Dongdong, Brown, Ben, wisk, Melchor, Álvaro Felipe, Schwartz, Tim, nsurbay, Komet, Meir, 2024. lief-project/LIEF.
- Zheng, Chaoyang, Wang, Yunchao, Huang, Huihui, et al., 2025. Survey of network protocol fuzzers: Taxonomy, techniques, and directions. *Comput. Secur.* 159, 104683. <http://dx.doi.org/10.1016/j.cose.2025.104683>.



Zhao Min Chen (asef18766) is currently a cybersecurity researcher at CyCraft Technology. He graduated from the Master's Program in Information Security at National Taiwan University, specializing in malware analysis, vehicle security and automated program analysis. He presented an academic poster at USENIX Security 2024 and has also delivered talks at events such as BlackHat USA, DEFCON, MOPCON and SITCON. In addition, he is an active CTF player, participating in teams such as Balsn, TWN48, and w33d for numerous CTF competitions. Notably, Mr. Chen has won 7th place in HITCON CTF 2023, 3rd place in DEFCON 31, and 7th place in DEFCON 32 represent Taiwan. He has also discovered several real-world vulnerabilities in Internet of Things and Bluetooth devices. He is continuously honing his skills in these fields.



Tien-Chih Lin (aka Dange) is the research team lead at CyCraft Technology, specializing in machine learning, red teaming, vehicle security, and cloud security. Alongside his technical expertise in offensive security, he has significant experience in cybersecurity management and holds a CACSP certification. As a co-founder of the Malware Village at DEFCON, he actively contributes to the global cybersecurity community. His research and work have been presented at conferences including Black Hat USA/Europe, HITCON CMT/ENT, USENIX Security, CYBERSEC, MOPCON, and ECCWS. Beyond research, Mr. Lin has delivered training at events such as AIS3, HITCON Training, and NICS. He is also one of the creators behind the educational cybersecurity board games “Cybercans” and “Cybercrete”.



Guan-Yan Yang (Graduate Student Member, IEEE) received the bachelor's degree from the Department of Information Management, National Dong Hwa University, Hualien, Taiwan, in 2022. He is currently pursuing a Ph.D. degree in the Department of Electrical Engineering at National Taiwan University, Taipei, Taiwan. In summer 2023, he was a Software Engineer Intern at the Design Technology Platform, Research and Development Division, Taiwan Semiconductor Manufacturing Company, and got First Place in the internship competition. His research interests include Security, Safety, Deep Learning, Generative AI, the Internet of Things (IoT), Formal Verification, and Software Testing. He has authored or co-authored more than 10 peer-reviewed



Yu-Sheng Lin (twinklestar03) is pursuing the B.S. degree in Computer Science from National Yang Ming Chiao Tung University from 2021. He served as a Security Research Intern at CyCraft AI Lab, Taipei, Taiwan, from April 2023 to August 2024. His research interests include cybersecurity, vulnerability detection, reverse engineering and blockchain. His recent research on fuzzing for Bluetooth Protocol was presented at USENIX Security 2024 and 2025. Mr. Lin is a member of \$(CyStick) Team in Taiwan and has participated in several CTFs. Mr. Lin has won 7th place in HITCON CTF 2023, 3rd place in DEFCON 31, and 7th place in DEFCON 32 represent Taiwan. He has also discovered several real-world vulnerabilities in Internet of Things.



Farn Wang (Member, IEEE) is a Full Professor at the Department of Electrical Engineering, National Taiwan University. He received the B.S. degree in Electrical Engineering from National Taiwan University in 1982 and the M.S. degree in Computer Engineering from National Chiao-Tung University in 1984. He completed his Ph.D. in Computer Science at the University of Texas at Austin in 1993. He is a founding member and chairperson of the Steering Committee of the International Symposium on Automated Technology for Verification and Analysis (ATVA) from 2003 to 2022, and has served on the ATVA advisory committee since 2022. He was also an Associate Editor of FMSD (International Journal on Formal Methods in System Design), Springer-Verlag. His research interests include formal verification, model-checking, software testing, security, verification automation, AI, and language models. He has been named a World's Top 2% Scientists in a career-long list by Stanford University since 2020.



Kuo-Hui Yeh (SM'16) serves as a professor at the Institute of Artificial Intelligence Innovation, National Yang Ming Chiao Tung University, Hsinchu, Taiwan. Prior to this appointment, he was a professor in the Department of Information Management at National Dong Hwa University, Hualien, Taiwan, from February 2012 to January 2024. Dr. Yeh earned his M.S. and Ph.D. degrees in Information Management from the National Taiwan University of Science and Technology, Taipei, Taiwan, in 2005 and 2010, respectively. He has contributed over 150 articles to esteemed journals and conferences, covering a wide array of research interests such as IoT security, Blockchain, NFC/RFID security, authentication, digital signatures, data privacy and network security. Furthermore, Dr. Yeh plays a pivotal role in the academic community, serving as an Associate Editor (or Editorial Board Member) for several journals, including the IEEE Transactions on Consumer Electronics (TCE), Journal of Information Security and Applications (JISA), Human-centric Computing and Information Sciences (HCIS), Symmetry, Journal of Internet Technology (JIT) and CMC-Computers, Materials & Continua. In the professional realm, Dr. Yeh is recognized as a Senior Member of IEEE and holds memberships with ISC2, ISA, ISACA, CAA, and CCISA. His professional qualifications include certifications like CISSP, CISM, Security+, ISO 27001/27701/42001 Lead Auditor, IEC 62443-2-1 Lead Auditor, and ISA/IEC 62443 Cybersecurity Expert, covering fundamentals, risk assessment, design, and maintenance specialties.